Homework Academic Integrity & AI Use Pledge

Course: _____

Homework / Assignment: _____

Name: _____

Student ID: _____

Student Pledge

I affirm that I have completed this assignment with academic integrity and that the submitted work represents my own understanding, judgment, and effort. I understand that **the use of generative AI tools, including ChatGPT, Claude, Gemini, and similar systems, is permitted in this course unless otherwise specified by the instructor or the assignment instructions.** However, AI should be used as a learning and productivity tool, not as a substitute for my own learning.

I therefore pledge that:

1. **I will not simply submit AI-generated content as my own work.**
   When I use AI to generate answers, explanations, code, designs, reports, or other content, I will review, revise, and integrate the material based on my own understanding.

2. **I will take responsibility for all submitted work, including AI-assisted work.**
   I understand that using AI does not transfer responsibility for the submitted work to the AI tool. I am responsible for its correctness, reasoning, quality, and integrity.

3. **I will make a genuine effort to understand the AI-generated content that I use.**
   This includes, when applicable, understanding the **code, algorithms, system architecture, mathematical reasoning, explanations, designs, or other content** generated or suggested by AI.

4. **I will independently verify AI-generated content before submitting it.**
   I will check for errors, unsupported claims, incorrect calculations, inappropriate assumptions, security or implementation issues, and other potential problems.

5. **I will modify and adapt AI-generated content when necessary.**
   I understand that meaningful use of AI may require correcting, rewriting, simplifying, extending, or otherwise adapting its output so that the final submission reflects my own understanding and meets the requirements of the assignment.

6. **I will be able to explain the work that I submit.**
   If requested, I will provide a written explanation, demonstration, presentation, or recorded explanation showing that I understand the submitted work and can explain the relevant **code, system, methodology, reasoning, or other AI-assisted content**.

7. **I will disclose my use of AI when required by the instructor or assignment.**
   When requested, I will identify the AI tool(s) used and describe how AI contributed to my work, including relevant prompts, outputs, or modifications when appropriate.

8. **I will use AI to support learning rather than circumvent it.**
   The purpose of homework is to develop my own knowledge, reasoning, problem-solving, and technical skills. I understand that simply obtaining an answer from AI without understanding the underlying work does not fulfill this purpose.

I understand that **AI-assisted work may be considered my own academic work only when I have appropriately reviewed, understood, verified, and, when appropriate, modified the AI-generated content in accordance with the requirements of this course.**

By signing below, I confirm that I have read, understood, and agreed to this pledge. I also acknowledge that I may be asked to demonstrate my understanding of any AI-assisted work that I submit.

Student Signature: _____

Date: _____

Homework 2: Turn Your MiniMind into an AI Agent with Memory

Release: [2026/10/30]; **Due:** [2026/11/13]; **Weight:** [10]% of final grade

1. Overview

In HW1 you trained a small language model. In this assignment you will turn a small model (≤128M parameters, e.g. your HW1 MiniMind model or the official minimind-3) into an **AI agent** that remembers what a user tells it across many conversations and can call tools.

An agent consists of four parts: a **model**, a **memory**, **tools**, and an **action loop** in which the model decides what to do next (save a fact, search memory, call a tool, or just answer). You will design how memories are written, retrieved, updated and refused, and train your model to use tools and to read its memory. Your agent will be evaluated on a multi-session memory benchmark in Traditional Chinese and ranked against your classmates.

Learning goals:

- Understand the components of an agent: model, memory, tools, and the action loop.
- Design memory write / read / update / abstention policies and understand their trade-offs.
- Use SFT or LoRA to teach a small model a fixed tool-call format, and experience the limits of small models.
- Use ablations to separate the contribution of framework design from that of model training.
- Observe catastrophic forgetting caused by agent fine-tuning (measured on your HW1 TMMLU+ held-out set).

2. Setup

1. Download `hw2_student_release.zip` from the course page. It contains the data (`data/train`, `data/val`), the evaluation script `eval_agent.py`, the agent base class `agent_base.py`, the tool definitions `tool_spec.json`, the data generator, and a starter file `my_agent.py`.
2. Install dependencies: `pip install torch transformers`.
3. Prepare a model in Hugging Face transformers format: your HW1 model (`model/` from your HW1 submission), or the official minimind-3 (`git clone https://huggingface.co/jingyaogong/minimind-3`).
4. Run the no-memory baseline to check your environment:

```
python eval_agent.py --model_dir <your_model> --agent agent_base:NoMemoryAgent --data_dir data/val
```

Compute: Evaluation on the validation set takes about 1–2 minutes on a single consumer GPU. SFT of a 64M model on the provided tool data takes well under an hour.

3. Data

The benchmark is synthetic and in Traditional Chinese. Each **persona** is a fictional user who talks to the agent over 8 sessions (about 128 user utterances, ~1,000 characters). Personal facts (where they live, their job, their children's ages, …) are scattered among small talk; some facts are later updated, and some sentences describe other people. After the conversation, the agent is asked 10 multiple-choice questions.

Split	Size	Permitted use
train	300 personas / 3,000 questions 3,000 tool-call items	✅ Training, any use
val	30 personas / 300 questions 200 tool-call items	✅ Local evaluation. Do not train on it.
test (private)	100 personas / 1,000 questions 300 tool-call items	❌ Held by the teaching staff. Uses phrasings, values and attributes that do not appear in train/val.

Question types (every type is reported separately):

Type	What it tests	Example
single	Recall of one fact across sessions	我叫什麼名字？
update	A newer fact overrides an older one (the old value is always an option)	我現在住在哪個城市？
multi	Combining two facts	我住在哪裡、做什麼工作？
abstain	The user never said it; the answer is 「你沒有提過」	我是什麼血型？
distractor	Similar facts about other people (「我朋友住高雄」) appear nearby	我住在哪裡？
tool	Retrieve numbers from memory, then compute	我兩個小孩的年紀加起來幾歲？

Every question has four options; except for multi questions, one option is always 「你沒有提過」. The tool-call set (`tool.jsonl`) gives a few user utterances and the expected action: `memory_save`, `memory_search`, `calculator`, `get_date`, or no call. See `README_data.md` for the full format.

Example. Persona `trp0001` in `train/memory.jsonl` talks over 8 sessions (~16 utterances each). Only the relevant utterances are shown below; the rest are small talk such as 「今天好冷喔」 or 「我剛洗完衣服」.

Session	User utterance	Annotation
1	我堂哥很喜歡游泳	other (about someone else; should not be saved)
2	我家女兒 4 歲了	fact: 女兒年齡 = 4
4	我在當工程師	fact: 職業 = 工程師
5	我最喜歡的運動是打籃球	fact: 運動 = 打籃球
6	提醒一下，我奇異果過敏	fact: 過敏 = 奇異果
6	我最近改成爬山了	update: 運動 = 爬山
6	我大學同學對芒果過敏	other
7	我住在台中	fact: 居住地 = 台中

Five of its ten questions (correct answer in bold):

Type	Question	Options	Why
update	我最喜歡的運動是什麼？	你沒有提過 / **爬山** / 慢跑 / 打籃球	打籃球 is the old value
distractor	我對什麼過敏？	**奇異果** / 你沒有提過 / 芒果 / 花生	芒果 and 花生 belong to other people
abstain	我大學念什麼系？	**你沒有提過** / 心理 / 資訊工程 / 法律	Never mentioned
multi	我住在哪裡、做什麼工作？	住台南，當工程師 / 住台中，當廚師 / **住台中，當工程師** / 住台南，當廚師	Both facts must be right
tool	我兩個小孩的年紀加起來幾歲？	20 歲 / 你沒有提過 / 19 歲 / **18 歲**	女兒 4 + 兒子 14

Each line of `memory.jsonl` is one persona (abridged):

```
{"persona_id": "trp0001",
```

```
 "profile": {"居住地": "台中", "運動": "爬山", ...},         # final correct facts
 "sessions": [{"session_id": 6, "date": "2026-03-13",
              "turns": ["...", "提醒一下，我奇異果過敏", ...],
              "annotations": [{"turn": 2, "type": "fact", "key": "過敏", "value": "奇異果
"}, ...]}, ...],
 "questions": [{"qid": "trp0001_q04", "type": "distractor", "question": "我對什麼過敏？",
               "options": {"A": "奇異果", "B": "你沒有提過", "C": "芒果", "D": "花生"},
               "answer": "A", "evidence": [[6, 2]]}, ...]}
```

Each line of `tool.jsonl` is one item; the last utterance is the one to respond to:

```
{"context": ["我家兒子 11 歲了"],
 "expected": {"action": "call", "tool": "memory_save", "args": {"key": "兒子年齡", "value":
"11"}}}
{"context": ["我上班是騎機車去的", "昨天睡得不太好", "我最近改開車上班了"],
 "expected": {"action": "call", "tool": "memory_save", "args": {"key": "通勤方式", "value":
"開車"}}}
{"context": ["我女兒 12 歲、兒子 6 歲，他們加起來幾歲？"],
 "expected": {"action": "call", "tool": "calculator", "args": {"expression": "12+6"}}}
{"context": ["今天走了好多路", "我朋友是廚師"],
 "expected": {"action": "answer"}}                  # about someone else: no tool call
```

Generating more data. You may run `gen_hw2_data.py` with other seeds, write your own templates, or generate data with a larger LLM, provided that every source is listed in your data manifest (Section 6). Because the private test set uses unseen phrasings, memorizing the public templates will not help; aim for diversity.

4. Agent Interface

Implement your agent in `my_agent.py` as a subclass of `BaseAgent`. **Do not modify** `eval_agent.py`, `agent_base.py`, or `tool_spec.json`; the staff will use the original versions.

Method	When it is called	What it should do
reset()	Before each persona	Clear all memory
new_session(date)	At the start of each session	Record the date if useful
chat(msg)	For every user utterance	Decide whether and what to store
prepare(question)	For every question	Return the memory text to put into the prompt (no options)
next_action(context)	For every tool-call item	Return the raw model output, including any <tool_call>

Method	When it is called	What it should do
save_snapshot() / load_snapshot()	After a persona / before each question	Save and restore memory so questions do not affect each other

Tool calls use MiniMind's native format:

```
<tool_call>
{"name": "memory_save", "arguments": {"key": "居住地", "value": "台南"}}
</tool_call>
```

Rules:

- Only the **last 300 characters** returned by `prepare()` are put into the prompt. Concatenating the whole history will not work; your agent must select what matters.
- Save only the user's own information; sentences about other people should not be saved.
- Main model: ≤ [128M] parameters. Auxiliary embedding model: ≤ [30M] parameters, or BM25 only.
- No external API calls. Memory must be stored locally.
- The full private evaluation must finish within [30] minutes on our environment.

5. Tasks

Task 1: Baselines (required)

Using your model, run the three reference agents in `agent_base.py` on the validation set: `NoMemoryAgent` (question only), `RecentContextAgent` (all history, truncated to the last 300 characters), and `KeywordRetrievalAgent` (store everything, retrieve by keyword overlap). These are your baselines.

Task 2: Build your memory agent (required)

Try to beat your baselines. Some directions to consider (you are not required to try all):

- **Write policy.** Store only important sentences, or let the model emit `memory_save` with a key-value pair. Saving the same key again overwrites the old value and directly handles update questions.
- **Retrieval.** BM25, a small embedding model, or let the model produce a `memory_search` query. Ordering and formatting within the 300-character budget matter.
- **Abstention.** When nothing relevant is found, make it easy for the model to choose 「你沒有提過」.
- **Computation.** For tool questions, look up the numbers and call the calculator inside `prepare()`, then put the result (e.g. 「計算結果：19 歲」) into the memory text.

- **Tool-call SFT.** The official minimind-3 already emits `<tool_call>` but almost always calls `memory_save` with wrong arguments. Use `data/train/tool.jsonl` to teach it when to save, search, compute, or not call a tool.
- **Reading SFT.** Even when the correct sentences are given, the official 64M model answers only about 41% correctly. Use `data/train/memory.jsonl` to build (memory + question → answer) training data.
- **Tricks for small models.** Special tokens, constrained decoding, scoring candidates instead of free generation, copying arguments from the user's sentence, and fallback rules when parsing fails.

Task 3: Report (required)

Write a report (max 4 pages) covering:

1. Your architecture (with a diagram) and the design decision for each memory operation: write, read, update, and abstention.
2. How you built your SFT data, with examples.
3. A results table on the validation set: baselines vs. each experiment, broken down by question type, plus the tool-call metrics.
4. Ablations, at least: no memory, keyword retrieval, your agent, and your agent with vs. without SFT.
5. TMMLU+ regression: your HW1 held-out TMMLU+ accuracy before and after agent fine-tuning.
6. Failure analysis: which question types are worst, and why (format error, not saved, wrong retrieval, …).
7. Training cost: GPU type, total GPU hours.

6. Submission

Submit a single archive `hw2_<student_id>.zip` containing:

```
<student_id>/
├── model/           # Hugging Face transformers format
├── code/            # my_agent.py, training scripts, data generation, run.sh
├── data_manifest.md  # Every dataset: source, size, how it was produced
├── logs/            # Training logs
└── report.pdf
```

Requirements:

- The model is loadable with `AutoModelForCausalLM.from_pretrained(path, trust_remote_code=True)`.
- Parameter limit: ≤ [128M] total parameters (main model).

- From inside `code/`, this command runs without error: `python eval_agent.py --model_dir ../model --agent my_agent:MyAgent --data_dir <data>`.
- The evaluation must finish within [30] minutes on our environment.
- `run.sh` must reproduce your final model from the data listed in your manifest.

7. Evaluation Protocol

Memory benchmark. For each persona, the evaluation script feeds every utterance to `chat()` and saves a snapshot. For each question, it restores the snapshot, calls `prepare(question)`, keeps the last 300 characters, and builds the prompt below with the model's chat template:

```
以下是你記得的關於使用者的資訊：
<your memory text>

問題：<question>
A. <option>
B. <option>
C. <option>
D. <option>
請直接回答選項字母。
答案：
```

Scoring. For each option o (tokens $t_1 \ldots t_k$, |o| characters), we compute

$$\text{score}(o) = (1/|o|) \times \Sigma_i \log P(t_i \mid \text{prompt}, t_1 \ldots t_{i-1})$$

i.e. the average log-probability per character of the option text following 「答案：」. The option with the highest score is the prediction. We do not score the letters A–D directly, because small models tend to pick the same letter for every question (in our test, minimind-3 picked A for all 300 validation questions).

Memory Accuracy is computed in two steps:

1. For each of the six question types, Acc(type) = (number of correct questions of that type) / (number of questions of that type).
2. Memory Accuracy = the average of the six Acc(type) values (macro-average), so every type counts equally regardless of how many questions it has. Random guessing scores about 25%.

Tool-call benchmark. For each item in `tool.jsonl`, the script calls `next_action(context)` and parses the output. An output **contains a call** if it includes `<tool_call>` (or `<call>`). A call is **parsed** if the text between the tags is valid JSON with a string `name` and an object `arguments`. Let

- N = number of items;
- N_call = number of items whose expected action is a call (the other items expect a plain answer);

- N_out = number of items whose output contains a call.

Metric	Formula	Meaning
Format validity	(# outputs that contain a call and are parsed) / N_out	When the model writes a tool call, can it be read?
Decision accuracy	(# items where contains-a-call = expected-a-call) / N	Did it call a tool exactly when it should?
Tool accuracy	(# call items, parsed, with the expected tool name) / N_call	Among items that need a call, did it pick the right tool?
Argument accuracy	(# call items with the right tool AND matching arguments) / N_call	Right tool and right arguments (rules below)
End-to-end	(# call items with argument match + # answer items with no call) / N	Fraction of items handled completely correctly

Argument matching rules (lenient on wording, strict on content):

Tool	Arguments match if
memory_save	key contains one of the accepted keywords of that attribute (e.g. 通勤 / 上班 / 交通), and value contains the expected value (e.g. 開車; 「11 歲」 also matches 11)
memory_search	query contains one of the accepted keywords (e.g. 過敏)
calculator	expression evaluates to the expected number (12+6 and 6+12 are both correct)
get_date	always (no arguments)

Worked example with real minimind-3 outputs:

Input	Expected	Model output	Result
我家兒子 11 歲了	memory_save(key=兒子年齡, value=11)	memory_save(query="10 歲女兒 資訊")	decision ✓ / tool ✓ / argument ✗ (no key, wrong value)
今天幾月幾號？	get_date()	memory_save(query="今天幾月幾號")	decision ✓ / tool ✗ / argument ✗

Input	Expected	Model output	Result
我朋友是廚師	no call	memory_save(...)	decision ✗ (should not call)

Reference results. Measured by the teaching staff using the official minimind-3 weights (64M, no additional training) on the validation set. These are not numbers published by MiniMind.

Agent	single	update	multi	abstain	distractor	tool	Memory Acc.
NoMemoryAgent	0.22	0.26	0.19	0.64	0.22	0.27	0.30
Oracle (staff only)	0.55	0.32	0.26	0.64	0.39	0.28	0.41

Oracle puts the annotated evidence sentences directly into the prompt, showing how well the untrained model reads its memory. Abstain is high for both because with no relevant memory the model tends to choose 「你沒有提過」.

Format validity	Decision acc.	Tool acc.	Argument acc.	End-to-end
0.56	0.63	0.22	0.00	0.06

minimind-3 already knows the `<tool_call>` format, but it calls `memory_save` for almost everything (hence decent decision accuracy), uses the wrong argument names, and copies whole sentences as arguments. Teaching it which tool to use and what to fill in is part of this assignment.

Leaderboard score. The leaderboard combines both benchmarks on the private test set:

```
Leaderboard Score = 0.7 × Memory Accuracy + 0.3 × Tool End-to-End
```

Memory remains the main part, while the tool term rewards agents whose model actually learns to call tools correctly. For reference, minimind-3 with NoMemoryAgent scores 0.7 × 0.30 + 0.3 × 0.06 ≈ 0.23 on the validation set. `eval_agent.py` prints this score at the end of every run.

All models will be run on the same evaluation environment.

8. Grading

Component	Weight	Criteria
Baselines	15%	Task 1 baselines run correctly; your agent passes eval_agent.py

Component	Weight	Criteria
Leaderboard	35%	Leaderboard Score (0.7 × Memory Accuracy + 0.3 × Tool End-to-End) on the private test set, graded in tiers (see below)
Reproducibility	15%	run.sh works; manifest (including any LLM-generated data) is complete; logs are consistent with the model
Report	35%	Clarity, quality of design, experiments and ablations, honest analysis

Leaderboard tiers. Scores within about 3 percentage points are treated as ties. Tiers: top 30% → 35 pts, next 30% → 30 pts, next 30% → 25 pts, remaining valid submissions → 20 pts. Beating the Leaderboard Score of your own KeywordRetrievalAgent baseline (same model) by at least 3 points earns 35 pts regardless of rank. If your TMMLU+ accuracy drops by more than 5 points after agent fine-tuning, explain why in the report.

9. Academic Integrity

- Discussing ideas with classmates is allowed; sharing code, model weights, or data pipelines is not.
- You may use AI coding assistants and LLM-generated training data; disclose how you used them in the report appendix and in your data manifest.
- Any attempt to obtain or use the private test set results in zero for the leaderboard and reproducibility components.

10. Resources

- MiniMind: https://github.com/jingyaogong/minimind
- minimind-3 (transformers format): https://huggingface.co/jingyaogong/minimind-3
- LongMemEval (long-term memory benchmark): https://arxiv.org/abs/2410.10813
- LoCoMo (very long-term conversational memory): https://arxiv.org/abs/2402.17753